

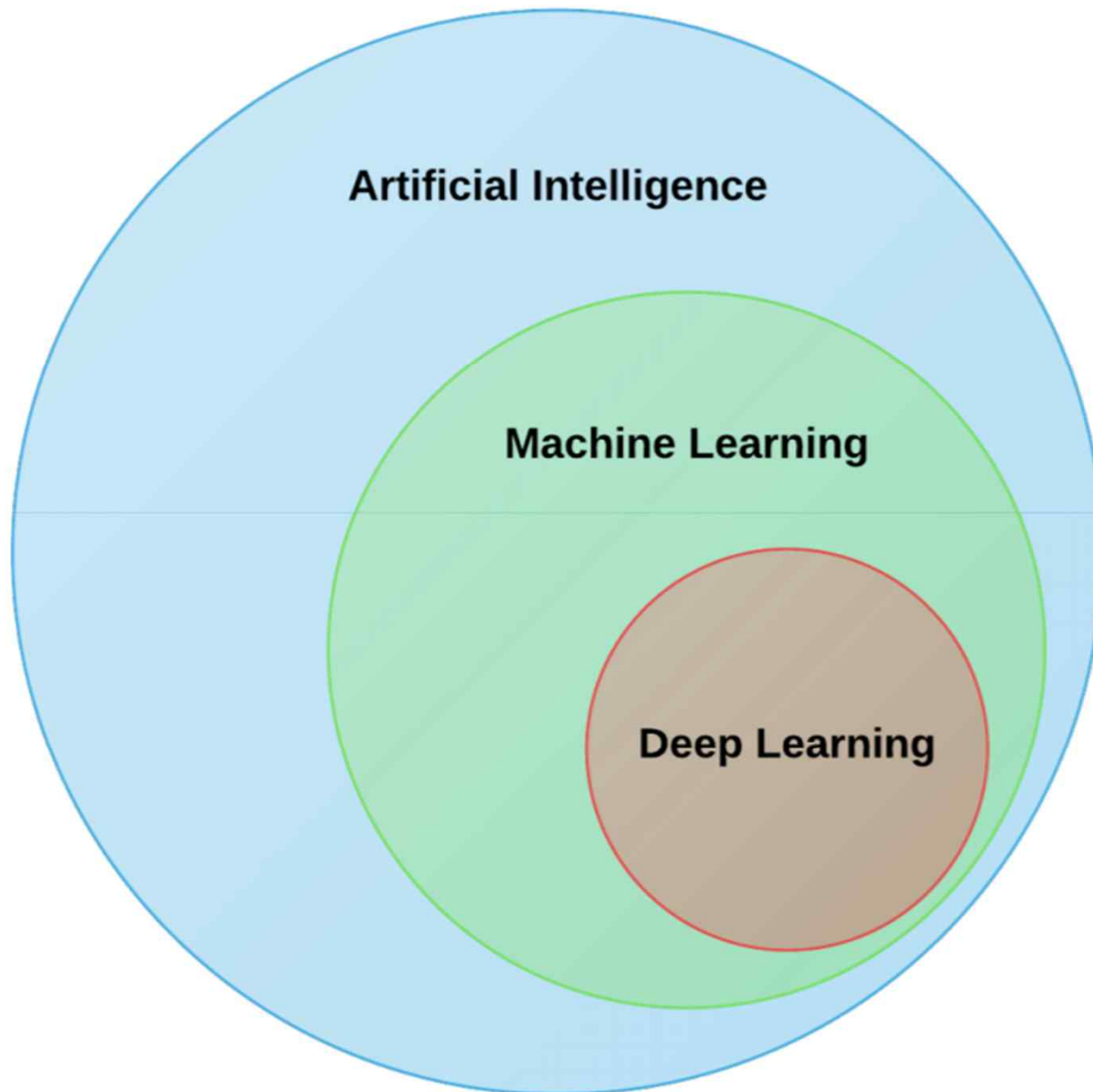
강화학습 기초

목차

- ✓ 강화학습의 개요
- ✓ 확률로 본 강화학습
- ✓ MDP(Markov Decision Procss)
- ✓ 가치함수, 큐함수, 정책
- ✓ 벨만 방정식
- ✓ Monte Carlo부터 Q-learning까지
- ✓ Frozen Lake

강화학습의 개요

인공지능의 분류



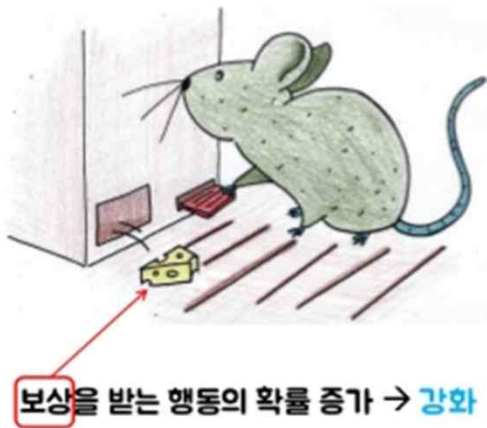
학습방법

- 지도학습
- 비지도학습
- 강화학습

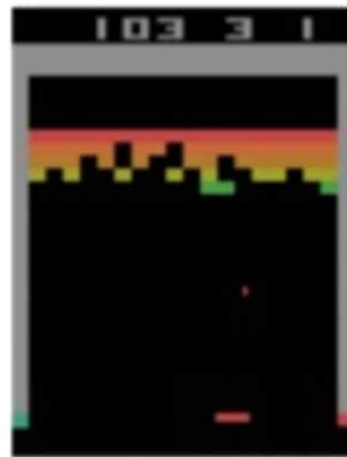
강화학습의 빠른 진화

- ✓ 강화학습의 방식은 범용적으로 활용할 수 있어, 새로운 환경에서 학습만 반복하면, 하나의 알고리즘을 가지고 다양한 환경에 적용 가능한 인공지능을 구현해 낼 수 있습니다.
- ✓ 일반인에게 인공지능의 혁신을 알렸던 알파고의 핵심 기술이 바로, 강화학습 기반이었습니다.

스키너의 쥐 실험



Atari Breakout



강화학습의 빠른 진화

- ✓ 알파고 이후 강화학습을 적용한 인공지능 연구가 본격화 되면서 바둑을 넘어 다양한 분야에 강화학습이 적용되며 혁신을 만들어 내고 있습니다.
- ✓ 하지만, 강화학습 기반 인공지능의 발전은 그 적용분야가 게임환경과 같은 2차원 가상환경에서의 지능 구현이었습니다.
- ✓ 때문에 실제 인간의 환경과는 큰 차이가 있어, 강화학습을 현실세계의 문제에 적용하는 것에는 한계가 있었습니다.
- ✓ 이러한 한계를 극복하기 위해 인공지능 분야의 주요 선도 연구 단체, 기업들은 3차원 환경 혹은 실제 물리적 환경에서 강화학습을 적용하기 위한 선행연구에 집중하기 시작했습니다.



Gym is a toolkit for developing and comparing reinforcement learning algorithms. It supports teaching agents everything from walking to playing games like Pong or Pinball.

[View documentation >](#)

[View on GitHub >](#)



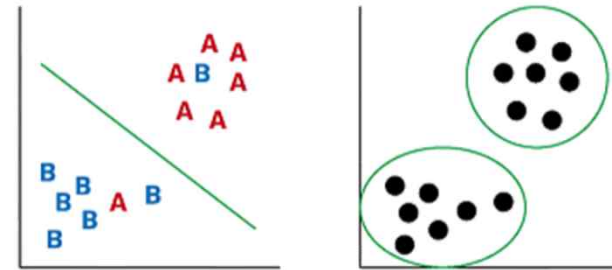
DeepMind

DQN(ATARI Breakout) -> AlphaGo -> StarCraft

기계학습(Machine Learning)의 분류

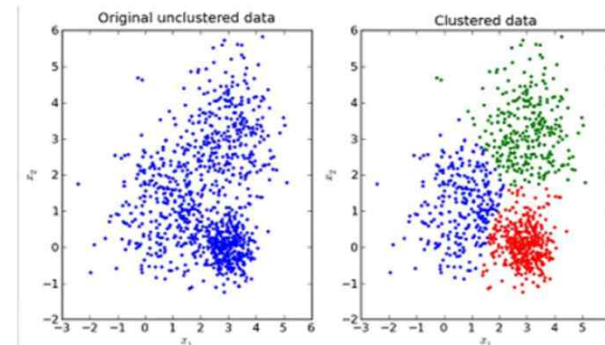
✓ Supervised Learning 지도학습

- Input 과 labels을 이용한 학습
- 분류(classification), 회귀(regression)
- $Y = f(x)$



✓ Unsupervised Learning 비지도 학습

- Input만을 이용한 학습
- 군집화(clustering), 압축(compression)
- $X \sim p(x)$ or $x = f(x)$



✓ Reinforcement Learning 강화학습

- Label 대신 reward가 주어짐
- Action selection, policy learning
- Find a Policy, $p(a|s)$
 - which maximizes the sum of reward



지도학습, 비지도학습과 강화학습 비교

✓ 지도학습. 비지도학습

- 학습 데이터의 순서가 달라져도 학습의 결과는 달라지지 않음.

✓ 강화학습

- 데이터의 관측 순서가 결과에 영향을 미치는 **순차적 의사결정 문제**를 다룸.
- Sequential Decision Making

✓ 강화학습은 마르코프 결정 과정(MDP) 을 거친다.

- 환경과 상호작용하는 에이전트가 현재 상태에서 행동을 선택하고 그 결과 상태가 전이되며 보상을 입력 받는 일련의 과정
- 순차적 의사결정 문제 수학적 표현 또는 수학적 정의

✓ 지도학습 수동적, 강화학습 능동적

- 감독학습 : 수동적으로 주어진 입력과 그에 대한 정답 (지도 신호)을 교사로부터 피드백 받는다.
- 강화학습 : 학습자가 행동을 능동적으로 생성해야 하며 그에 대해 보상치(평가 신호)만을 환경으로 부터 피드백으로 받는다.

지도학습과 강화학습 비교

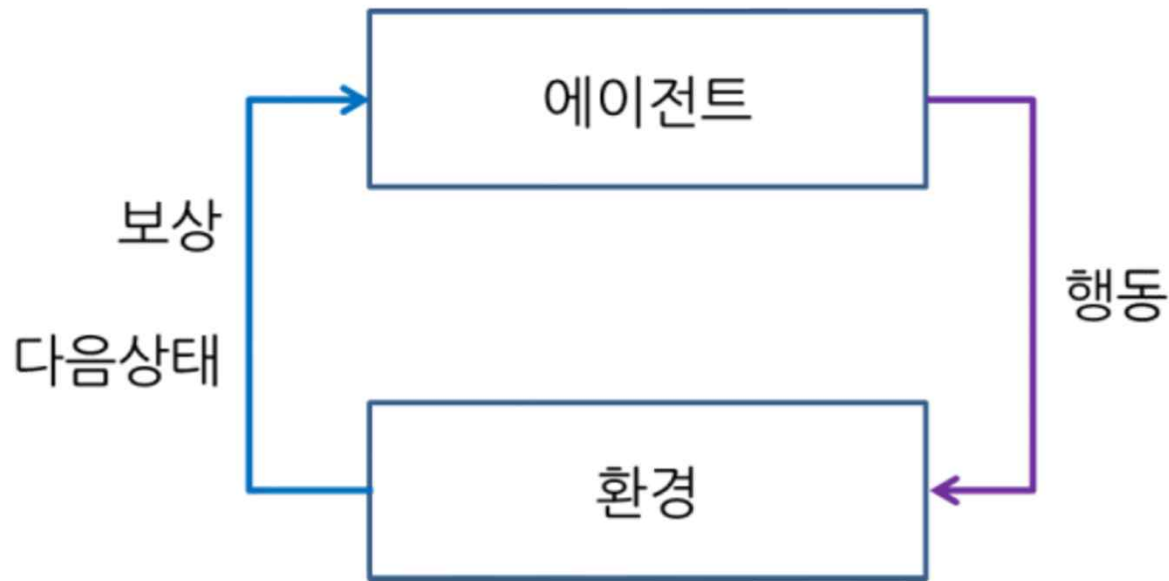
✓ 지도학습

- 학습자(에이전트)가 어떠한 행동을 선택할지 가르침을 받음

✓ 강화학습

- 가르침을 받지 아니하며 반드시 환경을 탐색하여 그 상황에서 수행한 행동에 따른 보상을 취득한다.
- 그 과정에서 강화학습에는 선택하기 까다롭지만 흥미로운 특성이 존재하는데, 행동은 현재 상태의 즉각적 보상에도 영향을 미칠 뿐만 아니라, 다음 상황에도 영향을 미쳐 결국 이후 모든 보상에 영향을 미치는 특성이다. (마르코프 과정)
- 즉 시행착오 (trial-and-error) 탐색과 지연 보상(delayed reward)이 강화학습을 다른 기계 학습과 구별짓는 중요한 특성 이다.

강화학습 모델



1. 에이전트가 자신의 상태를 관찰하여 특정 기준에 따라 행동을 선택함
2. 선택한 행동을 환경에서 실행 환경으로 부터 다음 상태와 보상을 받음
3. 보상을 통해 에이전트가 가진 정보를 수정함

- ✓ **에이전트** : 상태를 관찰, 행동을 선택, 목표지향
- ✓ **환경** : 에이전트를 뺀 나머지
- ✓ **상태** : 현재의 상황을 나타내는 정보
- ✓ **행동** : 현재상태에서 필요한 동작
- ✓ **보상** : 행동에 대한 보상

순차적 행동 결정 문제

✓ 순차적으로 행동을 결정하는 문제

- Sequential Decision Making
- $s_0, a_0, r_1, s_1, a_1, r_2, \dots s_T$

✓ 지도학습과 비지도학습에서는 학습 데이터의 순서가 달라져도 학습의 결과는 달라지지 않는다.

✓ 하지만 강화학습은 데이터의 관측 순서가 결과에 영향을 미치는 순차적 의사결정 문제를 다룬다.

✓ 수학적 표현 혹은 수학적 정의

- MDP(Markov Decision Process)
- 에이전트가 순차적 행동 방식 결정문제에 접근할 수 있게 합니다.

강화학습 요약 정리

✓ 불확실한 상황에서 의사결정 : "확률"에 기초하여 분석

- 확률적 과정(Stochastic Process) : 어떤 사건이 발생할 확률 값이 시간에 따라 변화해 가는 과정
- 마코프 과정(Markov Process) : 확률적 과정 중에서 한 가지 특별한 경우

✓ 마코프 과정

- 어떤 상태가 일정한 간격으로 변하고, 다음 상태는 현재상태에만 의존하며 확률적으로 변하는 경우의 상태의 변화
- 즉, 현재 상태에 대해서만 다음 상태가 결정되며, 현재 상태에 이르기까지의 과정은 전혀 고려할 필요가 없음.

✓ 마코프 연쇄(Markov Chain)

- 마코프 과정에서 연속적인 시간 변화를 고려하지 않고, 이산적인 경우만 고려한 경우
- 마코프 연쇄는 각 시행의 결과가 여러 개의 미리 정해진 결과 중의 하나가 되며, 각 시행의 결과는 과거의 역사와는 무관하며 오직 바로 직전 시행의 결과에만 영향을 받는 것이 특징이다.

✓ 강화학습을 푸는 가장 기본적인 방법 두 가지는 값 반복(Value Iteration)과 정책 반복(Policy Iteration)이다.

- 이를 설명하기 위해서는 먼저 값을 정의할 필요가 있다. 만약 우리가 특정 상태에서 시작했을 때, 얻을 수 있을 것으로 기대하는 미래 보상의 합을 구할 수 있다면 해당 함수를 매번 최대로 만드는 행동을 선택할 수 있을 것이고, 이렇게 최적의 정책 함수를 구할 수 있게 된다.

✓ 바로 이 미래에 얻을 수 있는 보상들의 합의 기대 값을 값 함수(Value Function)이라고 한다.

- 이 함수는 바로 현재 상태 뿐만 아니라 미래의 상태들, 혹은 그 상태에서 얻을 수 있는 보상을 구해야 하기 때문에 직관적으로 정의할 수는 없다. 일반적으로 강화학습에서는 이 값 함수를 구하기 위해 벨만 이퀄이션(Bellman Equation)을 활용한다.

강화학습에 사용된 확률

✓ 순서와 상관있다. 불확실하다.

- 불확실성 : 확률
 - Ω (표본) $\rightarrow$ X (확률변수) $\rightarrow$ $P(X)$ (확률분포함수)
- 불확실성, 순서(연속) : 확률 과정
 - Ω (표본) $\rightarrow$ X_t (확률과정) $\rightarrow$ $P(X_t)$ (확률분포함수)

✓ 바로 다음 미래는 현재에만 영향을 받는다.

- 마르코프 과정

✓ 이산시간에 대한 마르코프 과정

하나의 행동만 고려함

- 마르코프 체인

✓ 여러 행동에 대한 마르코프 과정

마르코프 의사 결정(MDP)

- 마르코프 의사 결정

✓ 100년 전에 이미 마르코프에 의해서 정의 되었다.

- 이를 푸는 방법에 따라 다이나믹과 강화학습으로 나눈다.

확률 부터 강화학습의 관계

확률 :

표본공간(Ω) > 확률변수(X) > 확률분포함수($P(X)$) > 확률과정함수($P(X_t)$) > 마르코프과정 > 마르코프 체인 > 마르코프의사결정

순차의사결정 방식(MDP : S A P R r)

$s \rightarrow a \rightarrow r \rightarrow s \rightarrow a \rightarrow r \rightarrow s \rightarrow a \rightarrow r \rightarrow s \rightarrow a \rightarrow r \rightarrow s \rightarrow a \rightarrow r \rightarrow s \rightarrow a \rightarrow r \rightarrow s \rightarrow a \rightarrow r \rightarrow e$

벨만방정식

Reward > return(G) > Value function(V) > Q function(Q) > Policy(π) > E-greedy

Planning :

Dynamic Program
Value Iteration/Policy Iteration

Sampling :

몬테카를로 제어(MC), 시간차 예측(TD)
SARSA, Q-learning

Planning

Dynamic program
Value Iteration/Policy Iteration

Learning

MC, TD,
SARSA, Q-learning

순차의사결정

MDP : S A P R r

벨만 방정식 : G $V(s)/Q(s)$ $\pi(s)$ e-greedy

확률

Ω , X , $P(X)$, X_t , $P(X_t)$,
마르코프 과정, 마르코프 체인, 마르코프 의사 결정

확률로 본 강화학습

순차적 행동 결정 문제를 정의한다.

Random(랜덤)/Stochastic(추계)

- ✓ 시간적으로 미리(사전에) 결과에 대해 정확히 예측, 정의할 수 없다는 의미
 - 단, 어떤 확률적 분포를 가질 수 있다는 통계적 규칙성은 있음 🖱 랜덤성
- ✓ `Random(무작위)` 및 `Stochastic(추계)`를 같은 의미로 씀
- ✓ 추계적(Stochastic) 이란?
 - 불확실성을 확률분포로 대신하고, 이로부터 추계적인 상황을 묘사함을 의미

MDP의 확률과의 관계

✓ 확률 변수

- 개별 사건에 대하여 확률함수
- 전체 사건에 대하여 확률 분포함수
- 이산확률변수
 - 기대값 (Expected Value), 분산 (Variance), 표준편차 (Standard Deviation)

✓ 확률 과정

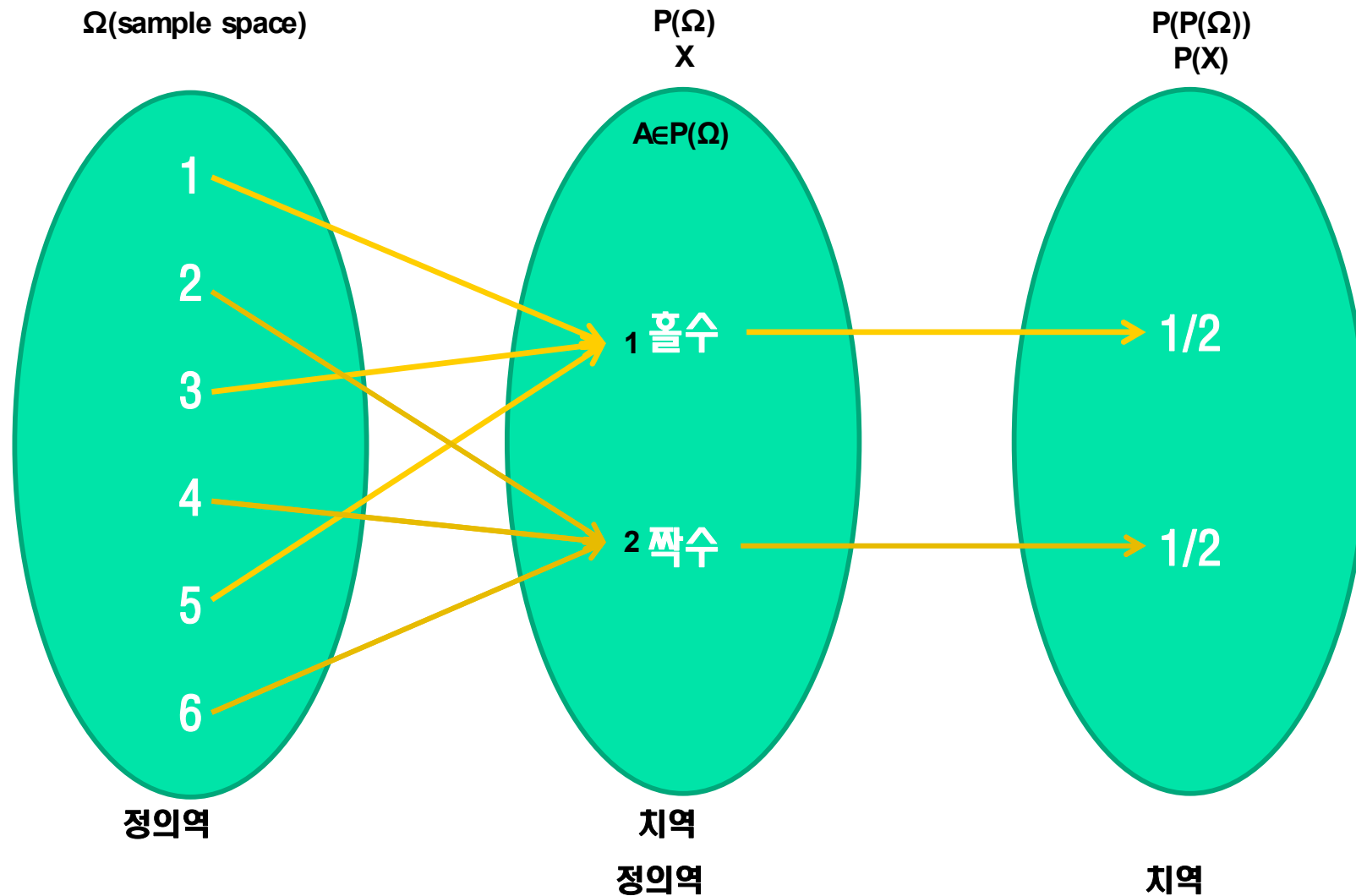
- 시간에 관련된 확률적인 성격을 갖는 계
 - 시간에 따라 확률도 변하게됨 (확률이론의 동적인 측면)
- Process(과정) - 시간을 고려한 상태를 말할 때는 주로 `과정`이라고 하고,
- 시간을 고려하지 않는 상태는 주로 `사건`이라고 함
- 따라서, **시간**에 따라 끊임없이 일어나는 **확률적** 현상을 의미함

✓ 결국, **랜덤**과정은,

- 순서가 부여될 수 있는 수많은 **확률변수**(즉, **확률변수 수열**)로 묘사될 수 있음. **시간**적으로 무수히 많은 **랜덤변수**를 다루고있다는 용어임
- **랜덤**과정이 어느 특정한 시각에 고정되면, . 이는, 하나의 **랜덤변수** $X(\xi)$ 가 됨
- **앙상블** : **랜덤**과정에 나타나는 시행 결과들의 모음/총체

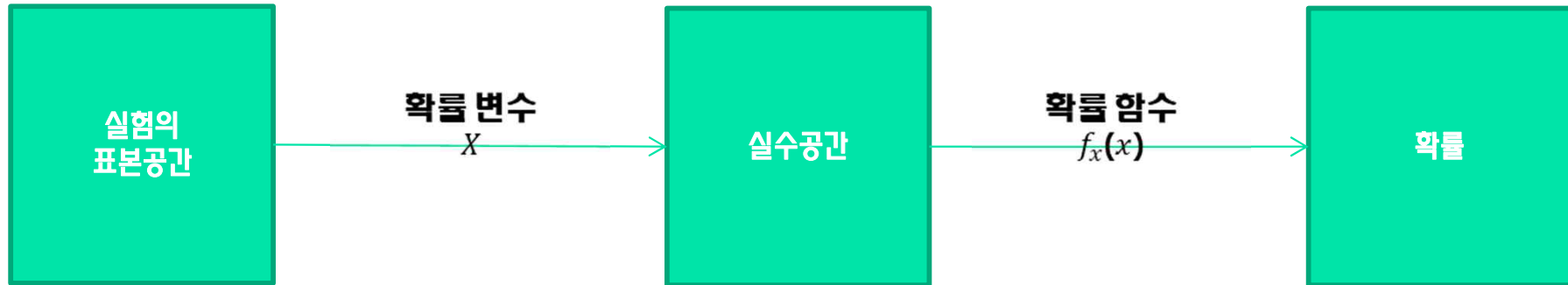
확률의 수학적 정의

확률은 표본이 아닌 사건을 입력으로 가지는 함수
사건(event) A 는 표본 공간 Ω 의 부분집합
 $A \subset \Omega \Leftrightarrow A \in \mathcal{P}(\Omega)$.



표본공간, 확률변수, 확률과정, 확률 함수

✓ $p : P(\Omega) \rightarrow [0, 1]$



확률변수에 대하여 정의된 실수들
0과 1사 이의 실수(확률)에 대응시키는 함수

✓ $p : P_t(\Omega) \rightarrow [0, 1]$



이산 확률 변수에서 기대값과 분산의 정의

- ✓ 확률변수라는 개념이 도입된 이후 “기대값” 과 “분산” 정의
 - 확률분포가 아니라 확률변수의 특징을 나타내는 값들
 - 기대값을 나타내는 표현 $E[X]$ 에는 확률변수 X 가 포함되는 것이 매우 자연스럽다.
- ✓ 확률함수 $p:P(\Omega)\rightarrow[0,1]$ 가 주어지고 이에 기반을 두는 확률 변수 $X:\Omega\rightarrow\mathbb{R}$ 가 주어지면 X 에 대한 기대값(expectation)을 다음과 같이 정의한다.
 - $(E(X)) = \sum_{a\in\Omega} p(x)X(a)$
 - 여기서 “기반으 둔다.” 는 것은 단순히 함수 p 와 X 의 정의구역이 Ω 로 일치한다는 뜻이다.

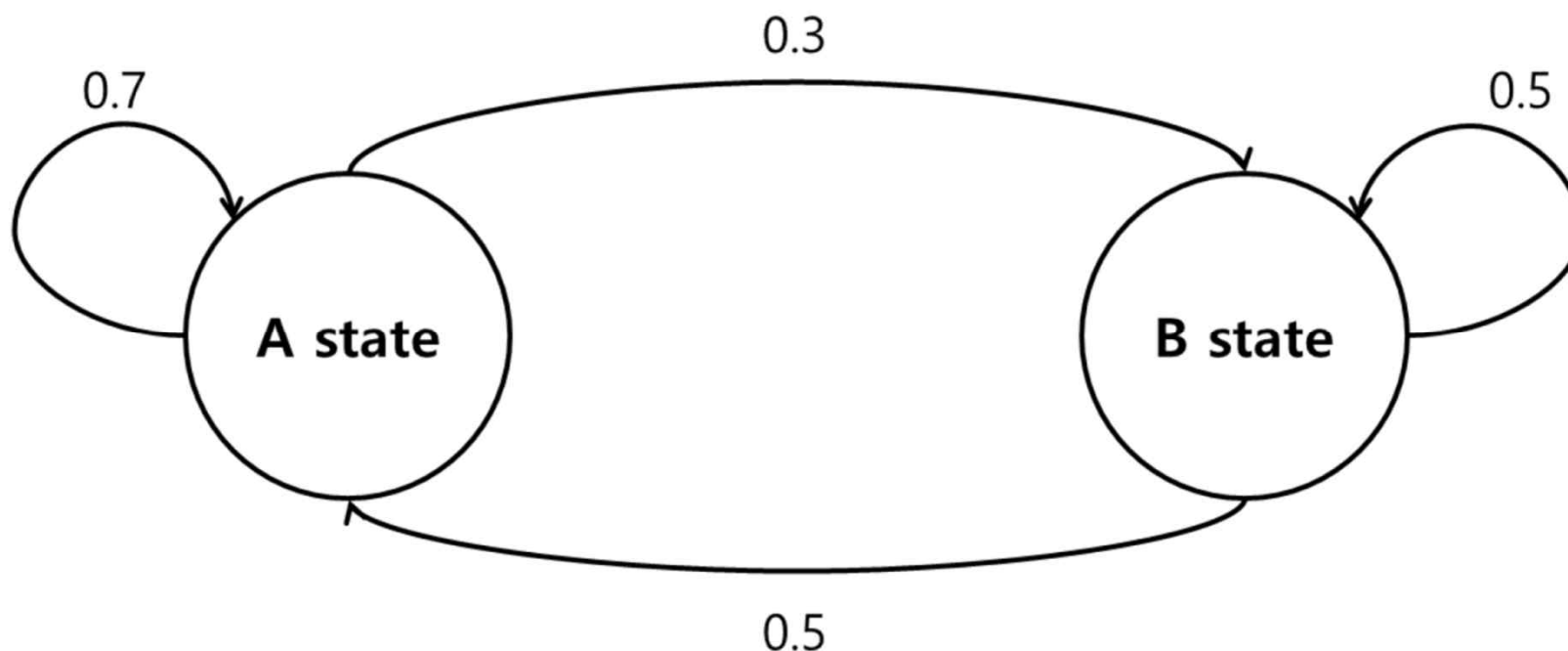
확률 프로세스

- ✓ 확률과정이란 시간축자를 갖는 확률변수들의 집합이다.
 - $\{Y, t=0, t=\pm 1, t=\pm 2, \dots\}$ 를 확률과정이라 할 때 관측된 시계열($y_1, y_2, \dots, y_m$)은 확률변수 $Y_1, Y_2, \dots, Y_m$ 의 실현된 값이다.
- ✓ 즉 Y_t 는 시점 t 에서의 확률변수를 나타내고 y_t 는 그 실현 값이다.
 - 특히 t 가 시간을 나타낼 때 이 확률과정을 시계열 과정 또는 시계열모형 이라고 한다.
- ✓ 확률과정과 실현 값의 차이는 횡단면 자료에서 모집단과 표본의 차이와 유사하다.
 - 모집단에 관한 추정을 위해 표본을 사용하는 것처럼 시계열에서는 내재하고 있는 확률과정에 관한 추정을 위해 실현값을 사용한다.

Markov Chain

- ✓ 확률론에서, 마르코프 연쇄(Markov chain)는 메모리를 갖지 않는 이산시간 확률 과정이다.
- ✓ 마르코프 연쇄는 시간에 따른 계의 상태의 변화를 나타낸다. 매 시간마다 계는 상태를 바꾸거나 같은 상태를 유지한다. 상태의 변화를 전이라 한다.

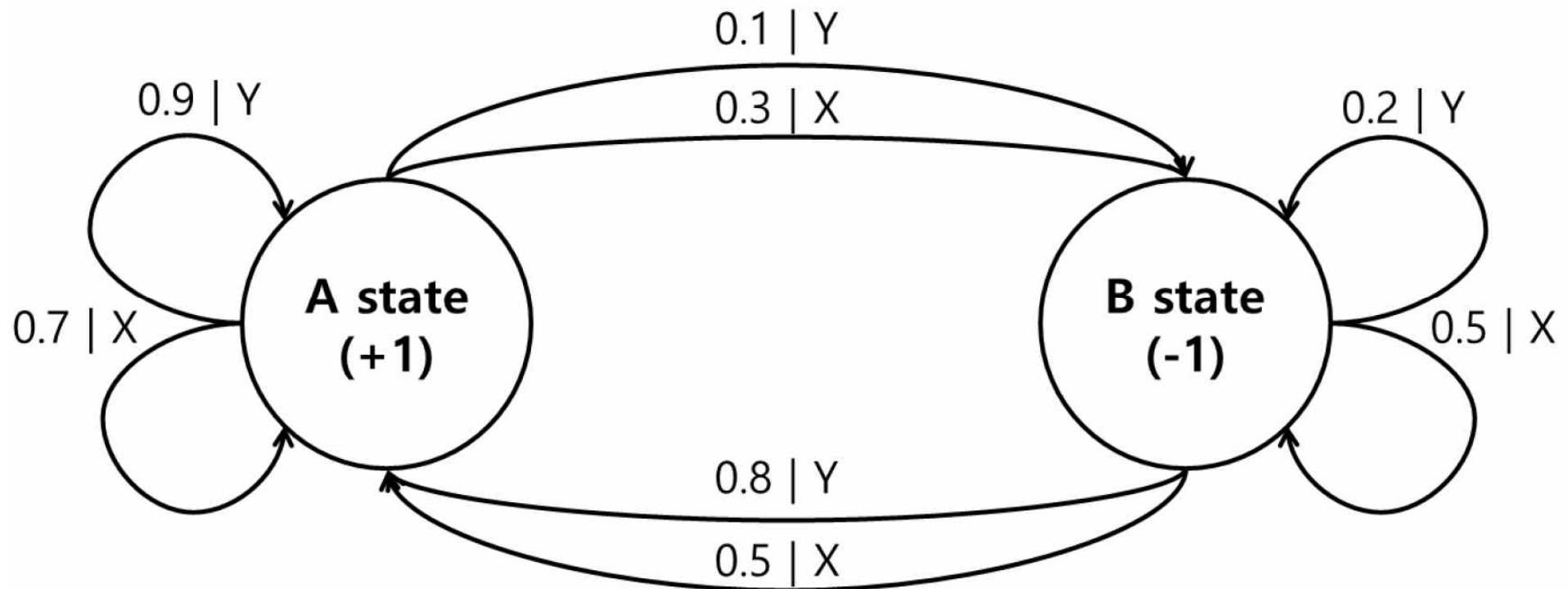
- Discrete state space : $S = \{ A, B \}$
- State transition probability : $P(S'|S) = \{P_{AA} = 0.7, P_{AB} = 0.3, P_{BA} = 0.5, P_{BB} = 0.5\}$
- Purpose : Finding a steady state distribution



- 마르코프 과정 . 다음 상태의 확률 값이 직전 과거에 만 종속되어있음 . (즉, 그 이전 과거의 역사와는 무관)

Markov Decision Process

- Discrete state space : $\mathbf{S} = \{ \mathbf{A}, \mathbf{B} \}$
- Discrete action space : $\mathbf{A} = \{ \mathbf{X}, \mathbf{Y} \}$
- (Action conditional) State transition probability : $\mathbf{P}(\mathbf{S}' | \mathbf{S}, \mathbf{A}) = \{ \dots \}$
- Reward function : $\mathbf{R}(\mathbf{S}'=\mathbf{A}) = +1$, $\mathbf{R}(\mathbf{S}'=\mathbf{B}) = -1$
- Purpose : Finding an optimal policy (maximizes the expected sum of future reward)



MDP

(Markov Decision Proccss)

순차적 행동 결정 문제를 정의한다.
상태, 행동, 보상 함수, 상태 변환 확률, 감가율

A Markov decision process (MDP) is a Markov reward process with decisions. It is an *environment* in which all states are Markov.

Definition

A *Markov Decision Process* is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{A}$ is a finite set of actions
- $\mathcal{P}$ is a state transition probability matrix,
 $\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$
- $\mathcal{R}$ is a reward function, $\mathcal{R}_s^a = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$
- γ is a discount factor $\gamma \in [0, 1]$.

상태(State)

- ✓ 상태(S_t) : 현재 상황을 나타낸다.
 - 환경이 에이전트에 제공해 준다.
 - 실물에서는 센서값을 상태로 봄
- ✓ 확률과정으로 나타냄
 - 어떤 t 에서 상태 S_t 는 정해져 있지 않는다.
 - 때에 따라서 $t=1$ 일 때 $S_t=1$ 일 수도 있고 $S_t=4$ 일 수도 있다.
- ✓ 시간 t 에서 확률변수 상태
 - $S_t=s$

Frozen Lake에는 16개의 상태가 있다.

S_0	F_1	F_2	F_3
F_4	H_5	F_6	H_7
F_8	F_9	F_{10}	H_{11}
H_{12}	F_{13}	F_{14}	G_{15}

행동(Action)

- ✓ 에이전트가 상태 S_t 에서 할 수 있는 가능한 행동의 집합 A
- ✓ $A_t = a$
 - 시간 t 에서의 행동 a
- ✓ $A = \{up, down, left, right\}$
 - 그리드월드에서 에이전트가 할 수 있는 행동의 집합
- ✓ 상태 변환 확률
 - 바람과 같은 예상치 못한 요소에 의해서 목적 상태에 도달하지 못하는 것을 고려 함

S	F	F	F
F	H	F	H
F	F	F	H
H	F	F	G

보상함수(Reward function)

✓ $R_s^a = E[R_{t+1}|S_t=s, A_t=a]$

✓ E 기대 함수 expectation value

□ 일종의 평균

✓ 상태 s에서 행동 a를 했을 경우에 받을 것이라 예상되는 숫자

✓ 보상함수를 기대 값으로 표현하는 것은

□ 환경에 따라서 같은 상태에서 같은 행동을 취하더라도 다른 보상을 줄 수도 있어서 이를 고려하기 위해

✓ Reward function

✓ $R_s^a = E[R_{t+1}|S_t=s, A_t=a]$

✓ | 조건문

□ | 뒤에 있는 것이 현재의 조건

✓ t+1

□ 행동하는 것은 t에서인데 보상을 받는 것은 t+1 이라는 것이다.

□ 보상을 agent가 알고 있는 것이 아니고 환경이 알려주기 때문이다.

□ 상태 S에서 행동 a를 하면 환경은 에이전트가 가게 되는 다음 상태 s' 과 에이전트가 받을 보상을 에이전트에게 알려준다.

□ 환경이 에이전트에게 알려주는 것이 t+1인 시점이다.

□ 따라서 에이전트가 받는 보상을 R_{t+1} 이라고 표현한다.

S, F : 0
H : -1
G : 1

S ₀	F ₀	F ₀	F ₀
F ₀	H ₋₁	F ₀	H ₋₁
F ₀	F ₀	F ₀	H ₋₁
H ₋₁	F ₀	F ₀	G ₁

상태 전이 확률(State transition probability)

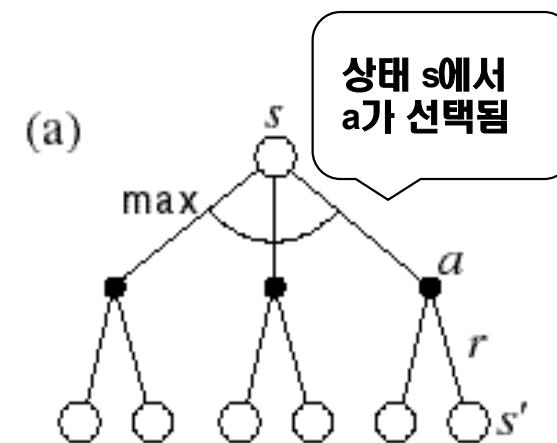
✓ $P_{ss'}^a = P[S_{t+1}=s' | S_t=s, A_t=a]$

- 에이전트가 상태 s 에서 행동 a 를 해서 s' 으로 가게 되는 확률
- 이 값은 보상과 마찬가지로 에이전트가 알지 못하는 값으로서 에이전트가 아닌 환경의 일부

✓ 상태 변환 확률은 환경 모델이라고 한다.

- 환경은 에이전트가 행동을 취하면 상태 변환 확률을 통해 다음에 에이전트가 갈 상태를 알려 준다.

S	F	F	F
F	H	F	H
F	???	F	H
H	F	F	G



감가율(Discount vector)

✓ 시간에 따라서 감가하는 비율을 감가율이라고 한다.

✓ 감가율의 정의

□ $\gamma \in [0, 1]$

□ 감가율 γ 0과 1 사이의 값이다.

✓ 감가율을 고려한 미래 보상의 현재 가치

□ $\gamma^{k-1} R_{t+k}$

□ 현재부터 시간이 k 만큼 지났기 때문에 미래에 받을 보상 R_{t+k} 는 γ^{k-1} 만큼 감가된다.

□ 더 먼 미래에 받을 수록 에이전트가 받는 보상의 크기는 줄어든 것이다.

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + R_{t+4} + R_{t+5} + \dots$$

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \gamma^4 R_{t+5} + \dots$$

S ₁	F ₁	F ₁	F
F	H	F ₁	H
F	F	F ₁	H
H	F	F ₁	G

S ₉ ^{0.5904}	F ^{0.6561}	F _{0.729}	F
F	H	F _{0.81}	H
F	F	F _{0.9}	H
H	F	F ₁	G

가치함수, 큐함수, 정책

순차적 행동 결정 문제를 푼다

Planning : Exhaustive Search, Dynamic Programming

Learning : Monte Carlo, TD, SARSA, Q-learning

가치함수, 큐함수, 정책 관련 내용 정리



R

G

V/Q

π

✓ 시간 t에서 행동을 통하여 받을 보상의 합(G)

$$\square R_{t+1} + R_{t+2} + R_{t+3} + R_{t+4} + R_{t+5} + \dots$$

✓ 감가율을 적용한 보상의 합(G)

$$\square G_t = R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 + \gamma^4 R_5 + \dots$$

$$\square = \sum_{k=1}^{\infty} \gamma^{k-1} R_{t+k}$$

✓ 가치 함수

$$\square v(s) = E[G_t | S_t = s]$$

✓ 큐함수

$$\square q(s) = E[G_t | S_t = s, A_t = a]$$

✓ 정책

$$\square \pi(s) = P[A_t = a | S_t = s]$$

$$\square \pi'(s) = \operatorname{argmax}_a q_{\pi}(s, a)$$

가치 함수, 큐함수

✓ 가치 함수

- 현재 상태에서 미래에 받을 보상의 합에 대한 기대값
 - 기대값을 취하는 이유는 현재상태에는 여러 액션이 있기 때문에 각 액션에 따른 미래에 대하여 받을 보상의 합에 대한 평균을 말하고 있다.

✓ 큐 함수

- 현재 상태에서 행동을 취한 경우 미래에 받을 보상의 합에 대한 기대값
 - 기대값을 취하는 이유는 액션에 따라 상태전이 확률이 있기 때문에 각 각 상태에 따른 미래에 대하여 받을 보상의 합에 대한 평균을 말하고 있다.
- 행동을 결정할 때도 확률이 있고 결정된 행동에서 상태로 갈 때도 확률이 있다.

V_tbl[16]로 할당됨

S _{V(0)}	F _{V(1)}	F _{V(2)}	F _{V(3)}
F _{V(4)}	H _{V(5)}	F _{V(6)}	H _{V(7)}
F _{V(8)}	F _{V(9)}	F _{V(10)}	H _{V(11)}
H _{V(12)}	F _{V(13)}	F _{V(14)}	G _{V(15)}

Q_tbl[16][4]로 할당

Q(0,0) S	Q(0,1) F	Q(0,2) F	Q(0,3) F
Q(1,0) F	Q(1,1) F	Q(1,2) F	Q(1,3) F
Q(2,0) F	Q(2,1) F	Q(2,2) F	Q(2,3) F
Q(3,0) F	Q(3,1) F	Q(3,2) F	Q(3,3) F
Q(4,0) F	Q(4,1) F	Q(4,2) F	Q(4,3) F
Q(5,0) H	Q(5,1) H	Q(5,2) F	Q(5,3) F
Q(6,0) F	Q(6,1) F	Q(6,2) F	Q(6,3) F
Q(7,0) H	Q(7,1) H	Q(7,2) F	Q(7,3) F
Q(8,0) F	Q(8,1) F	Q(8,2) F	Q(8,3) F
Q(9,0) F	Q(9,1) F	Q(9,2) F	Q(9,3) F
Q(10,0) F	Q(10,1) F	Q(10,2) F	Q(10,3) F
Q(11,0) H	Q(11,1) H	Q(11,2) F	Q(11,3) F
Q(12,0) H	Q(12,1) F	Q(12,2) F	Q(12,3) F
Q(13,0) F	Q(13,1) F	Q(13,2) F	Q(13,3) F
Q(14,0) F	Q(14,1) F	Q(14,2) F	Q(14,3) F
Q(15,0) G	Q(15,1) G	Q(15,2) F	Q(15,3) F

Value Function VS Q- Function

Value function : State만 고려

Definition

The state-value function $v_{\pi}(s)$ of an MDP is the expected return starting from state s , and then following policy π

$$v_{\pi}(s) = \mathbb{E}_{\pi} [G_t \mid S_t = s]$$

Q-function : State와 Action을 같이 고려

Definition

The action-value function $q_{\pi}(s, a)$ is the expected return starting from state s , taking action a , and then following policy π

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} [G_t \mid S_t = s, A_t = a]$$

Value function은 각 action에 확률과 Q-function값의 합

$$v_{\pi}(s) = \sum_{a \in A} \pi(a \mid s) q_{\pi}(s, a)$$

✓ 정책을 모든 상태에서 에이전트가 할 행동

- 상태가 입력으로 들어오면 행동을 출력으로 내보내는 일종의 함수
- 정책은 각 상태에서 단 하나의 행동만을 나타낼 수도 있고 확률적으로 $a1 = 10\%$, $a2 = 90\%$

✓ 최적 정책

- 에이전트가 강화 학습을 통하여 학습해야 하는 것

$$\pi(a | s) = P[A_t = a | S_t = s]$$

$$\pi'(s) = \operatorname{argmax}_a q_{\pi}(s, a)$$

벨만 방정식

벨만방정식이라고 쓰고 다이나믹 프로그래밍이라 부른다.

Planning – model base

가치함수의 벨만 방정식

✓ 현재 상태에서 미래의 받을 보상의 합에 대한 $\mathbb{E}$

$$V(s) = \mathbb{E}[G_t \mid S_t = s]$$

$$\mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

$$\mathbb{E}[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) \mid S_t = s]$$

$$\mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s]$$

$$\mathbb{E}[R_{t+1} + \gamma V(S_{t+1}) \mid S_t = s]$$

$$V^\pi(s) = \sum_a \pi(s, a) \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^\pi(s')]$$

V_tbl[16]로 할당됨

S G_t	F G_{t+1} R_{t+1}	F G_{t+2} R_{t+2}	F $V(3)$
$V(4)$ F	H $V(5)$	F G_{t+3} R_{t+3}	H $V(7)$
$V(8)$ F	F $V(9)$	F G_{t+4} R_{t+4}	H $V(11)$
$V(12)$ H	F $V(13)$	F G_{t+5} R_{t+5}	G $V(15)$

$$\begin{aligned}
 G_t &= r_0 R_{t+1} + r_1 R_{t+2} + r_2 R_{t+3} + r_3 R_{t+4} + r_4 R_{t+5} = R_{t+1} + r G_{t+1} \\
 G_{t+1} &= r_0 R_{t+2} + r_1 R_{t+3} + r_2 R_{t+4} + r_3 R_{t+5} = R_{t+2} + r G_{t+2} \\
 G_{t+2} &= r_0 R_{t+3} + r_1 R_{t+4} + r_2 R_{t+5} = R_{t+3} + r G_{t+3} \\
 G_{t+3} &= r_0 R_{t+4} + r_1 R_{t+5} = R_{t+4} + r G_{t+4} \\
 G_{t+4} &= r_0 R_{t+5} = R_{t+5}
 \end{aligned}$$

큐함수의 벨만방정식

$$q(s) = E[G_t \mid S_t = s, A_t = a]$$

$$E[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s, A_t = a]$$

$$E[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) \mid S_t = s, A_t = a]$$

$$E[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a]$$

$$E[R_{t+1} + \gamma q(S_{t+1}) \mid S_t = s, A_t = a]$$

$$q^\pi(s) = \sum_a \pi(s, a) \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma q^\pi(s')]$$

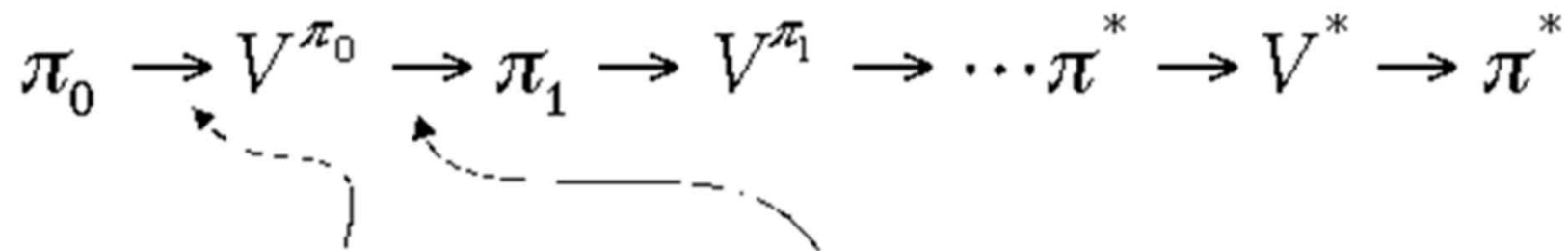
Policy Iteration

- ✓ Policy iteration consists of two simultaneous, interacting processes.
- ✓ Policy evaluation (Iterative formulation):
 - ▣ First, make the value function more precise with the current policy.
- ✓ Policy improvement (Greedy selection):
 - ▣ Second, make greedy policy greedy with respect to the current value function.
- ✓ Policy Iteration = Policy Evaluation + Policy Improvement

$$\begin{aligned}v_{\pi}(s) &\doteq \mathbb{E}_{\pi}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s] \\&= \mathbb{E}_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s] \\&= \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_{\pi}(s')],\end{aligned}$$

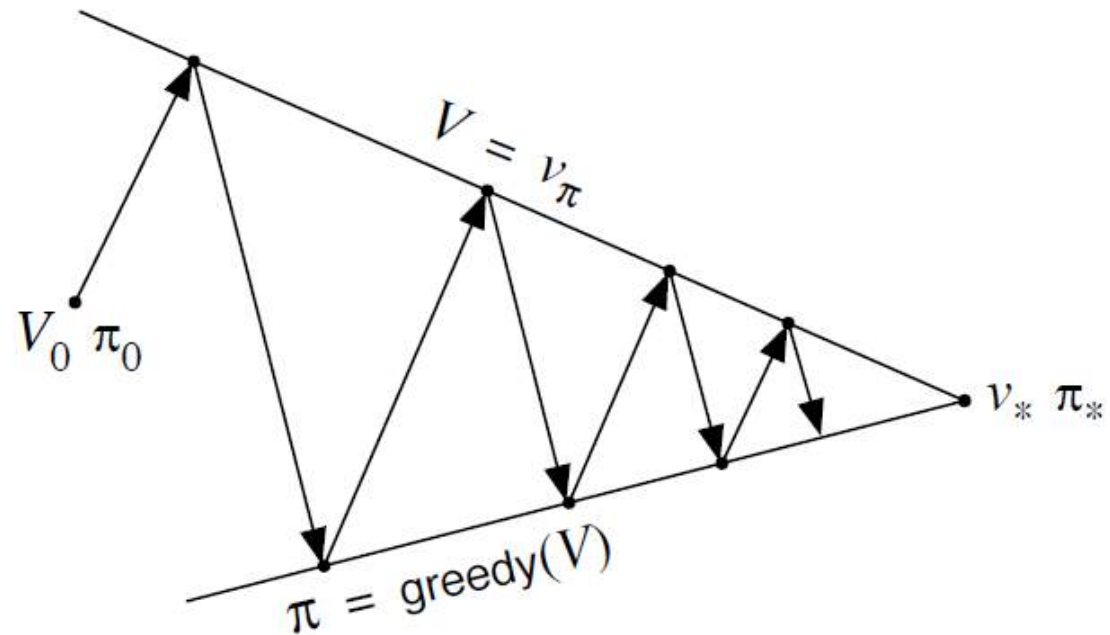
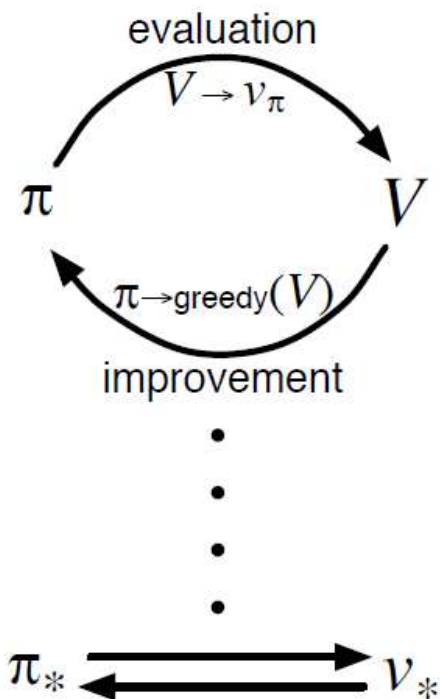
$$\begin{aligned}\pi'(s) &\doteq \operatorname{argmax}_a q_{\pi}(s,a) \\&= \operatorname{argmax}_a \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s, A_t = a] \\&= \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma v_{\pi}(s')],\end{aligned}$$

Policy Iteration



policy evaluation

policy improvement
“greedification”



Monte Carlo부터 Q-learning까지

Sampling이라고 쓰고 learning 이라고 읽는다.

Model free

여기서 부터 강화학습

Monte Carlo

$$\begin{aligned}V_{n+1} &= \frac{1}{n} \sum_{i=1}^n G_i = \frac{1}{n} (G_n + \sum_{i=1}^{n-1} G_i) \\V_{n+1} &= \frac{1}{n} (G_n + (n-1) \frac{1}{n-1} \sum_{i=1}^{n-1} G_i) \\V_{n+1} &= \frac{1}{n} (G_n + (n-1)V_n) \\V_{n+1} &= \frac{1}{n} (G_n + nV_n - V_n) \\V_{n+1} &= V_n + \frac{1}{n} (G_n - V_n) \\V_{n+1} &= V_n + \alpha (G_n - V_n)\end{aligned}$$



$$\sum_{i=1}^n G_i$$

$$\checkmark \quad V_{n+1} = \frac{1}{n} (G_1 + G_2 + G_3 + \cdots + G_{n-2} + G_{n-1} + G_n)$$

$$\sum_{i=1}^{n-1} G_i$$

$$(n-1) \frac{1}{(n-1)} \sum_{i=1}^{n-1} G_i$$

$$V_n = \frac{1}{(n-1)} \sum_{i=1}^{n-1} G_i$$

$$(n-1)V_n$$

$$= \frac{1}{n} ((n-1)V_n + G_n) = \frac{1}{n} (nV_n - V_n + G_n) = \frac{1}{n} (nV_n + G_n - V_n) = V_n + \frac{1}{n} (G_n - V_n)$$

$$= V_n + \alpha (G_n - V_n)$$

Monte Carlo 예측

- ✓ 다이내믹 프로그래밍에서 강화학습으로 넘어가는 가장 기본적인 아이디어
- ✓ 기대값을 샘플링을 통한 평균으로 대체하는 기법
- ✓ 에피소드를 하나 진행하고 에피소드 동안 지나온 상태의 반환값을 구함
- ✓ 이 반환값은 하나의 샘플이 되어서 각 상태의 가치함수를 업데이트 함

Initialize:

$\pi \leftarrow$ policy to be evaluated

$V \leftarrow$ an arbitrary state-value function

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Repeat forever:

Generate an episode using π

For each state s appearing in the episode:

$G \leftarrow$ return following the first occurrence of s

Append G to $Returns(s)$

$V(s) \leftarrow \text{average}(Returns(s))$

Temporal-Difference Learning 시간차 예측

- ✓ 타임스텝마다 가치함수를 업데이트함
- ✓ 벨만기대 방정식을 이용하여 가치함수를 업데이트 한다.

```
Input: the policy  $\pi$  to be evaluated
Initialize  $V(s)$  arbitrarily (e.g.,  $V(s) = 0, \forall s \in \mathcal{S}^+$ )
Repeat (for each episode):
  Initialize  $S$ 
  Repeat (for each step of episode):
     $A \leftarrow$  action given by  $\pi$  for  $S$ 
    Take action  $A$ , observe  $R, S'$ 
     $V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$ 
     $S \leftarrow S'$ 
  until  $S$  is terminal
```

E-greedy

- ✓ 일정한 확률로 랜덤하게 행동 선택

탐욕정책 $\pi'(s) = \operatorname{argmax}_a q_{\pi}(s, a)$

ε - **탐욕정책** $\pi(s) = \begin{cases} a^* = \operatorname{argmax}_a q(s, a), & 1 - \varepsilon \\ a \neq a^*, & \varepsilon \end{cases}$

SARSA 제어 – 시간차 제어

- ✓ 행동을 선택할 때 ϵ -탐욕 정책 사용 -> 가치함수를 사용하지 않고 큐함수 사용
 - 가치함수는 환경모델을 알아야함
- ✓ 하나의 샘플로 (s, a, r, s', a') 필요함
- ✓ 온폴리시 강화학습

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$

Repeat (for each episode):

Initialize S

Choose A from S using policy derived from Q (e.g., ϵ -greedy)

Repeat (for each step of episode):

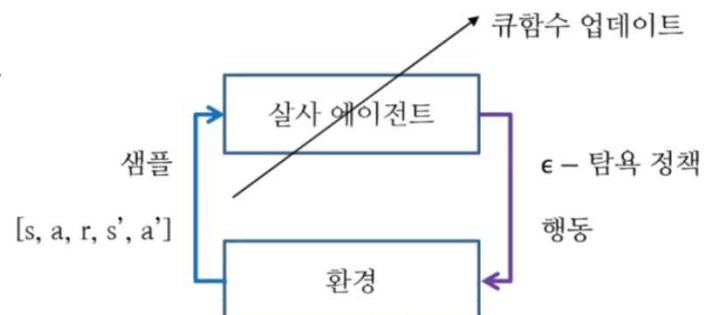
Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ϵ -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

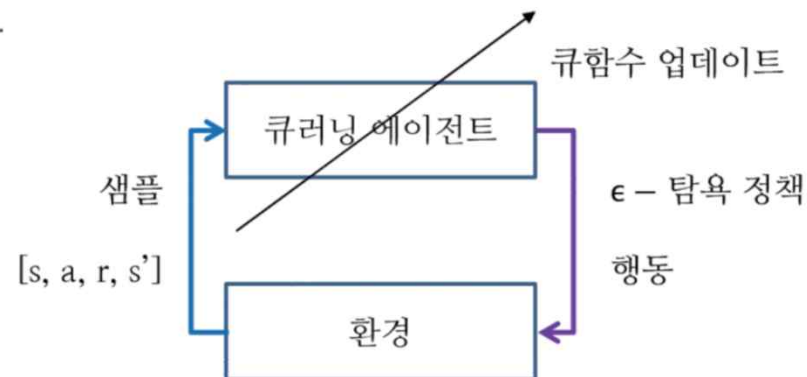
until S is terminal



Q-learning

- ✓ 오프폴리시 강화학습
- ✓ 행동을 선택할 때 ϵ -탐욕 정책 사용
- ✓ 큐함수 업데이트에는 벨만 최적 방정식을 이용한다.
 - 다음큐 함수중에서 가장값이 큰 큐함수를 이용해서 현재 큐함수를 업데이트

```
Initialize  $Q(s, a)$  arbitrarily
Repeat (for each episode):
  Initialize  $s$ 
  Repeat (for each step of episode):
    Choose  $a$  from  $s$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
    Take action  $a$ , observe  $r, s'$ 
     $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ 
     $s \leftarrow s'$ ;
  until  $s$  is terminal
```



몬테카를로 예측

$$V(s) \leftarrow V(s) + \alpha(G(s) - V(s))$$

시간차 예측

살사

E-greedy

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

큐러닝

E-greedy

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t))$$

Online video lectures

- **A Tutorial on Reinforcement Learning,**

<https://simons.berkeley.edu/talks/tutorialreinforcement-learning> 2017

- **Berkeley CS 294: Deep Reinforcement Learning, Spring 2017**

<http://rll.berkeley.edu/deeprlcourse/>, 2017

- **MIT 6.S094: Deep Learning for Self-Driving Cars (Lecture 2)**

<http://selfdrivingcars.mit.edu/>, 2017

- **Deep Reinforcement Learning (John Schulman, OpenAI)**

<https://www.youtube.com/watch?v=PtAlh9KShjo&t=2457s> (summary) and

https://www.youtube.com/watch?v=aUrX-rP_ss4&list=PLjKEIQIKCTZYN3CYBlj8r58SbNorobqcp (4 lectures)

- **UCL, David Silver, Reinforcement Learning**

<http://www0.cs.ucl.ac.uk/staff/d.silver/web/Teaching.html>, 2015

- **Stanford Andrew Ng CS229 Lecture 16**

<https://www.youtube.com/watch?v=Rtxl449ZjSc>, 2008



감사합니다.

e2g1234@naver.com
